

Project Report

CSC/ECE 591: Software for Robotics
NC State University • Spring 2026

Project Title: Evaluating Collaborative vs. Standalone Robot Navigation in Natural Disaster Environments

Team Name: *BGM*

Team Members: Rachit Gupta, Pranav Bhagwat, Rameez Malik

Date: April 24, 2026

GitHub: Course Project Repository

Abstract

Rapid deployment of autonomous systems is critical for search and rescue operations during natural disasters, where dynamically changing landscapes present significant navigational challenges. This report provides a comparative study between a collaborative UAV-UGV system versus a standalone UGV with SLAM in a low fidelity environment to understand how these system interact and perform at completing the same task. Utilizing PX4-ROS2 for aerial reconnaissance, the UAV maps the disaster zone from a high altitude and relays global map data to a Nav2-equipped UGV for optimized ground-level path planning. To evaluate the system's efficacy, we conduct a comparative analysis measuring the time-to-goal performance of the UAV-assisted UGV against a standalone UGV reliant strictly on real-time Simultaneous Localization and Mapping (SLAM). The results demonstrate that integrating prior aerial knowledge significantly reduces overall travel time for the UGV by eliminating the need for exploratory ground mapping and preventing route dead-ends. Ultimately, this collaborative UAV-UGV architecture provides a more efficient and reliable autonomous navigation strategy for time-critical disaster relief operations.

Introduction

2.1 Motivation

This project aims to develop and study UAV-UGV systems for emergency response to natural disasters. In a natural disaster situation, quick response is critical to saving human lives. One of the main challenges involved in providing relief is the dynamically changing landscape in case of a flood or earthquake, where paths can be blocked while the operation is underway.

To tackle this, the UAV (Unmanned Aerial Vehicle) maps the area from a higher altitude and relays the information to the UGV (Unmanned Ground Vehicle) that navigates and reaches the destination. The system uses PX4-ROS2 for UAV control and Nav2 for the UGV. We compare the performance (measured in time to reach the goal) with a UGV that does not have prior information of the area and needs to use SLAM (Simultaneous Localization and Mapping) to reach its destination to check the advantages of the UAV in these scenarios.

2.2 Individual Contribution

Since this project is done by a team of 3 students, the tasks were divided such that each team member would be responsible for a single robot as follows:

- Pranav Bhagwat

He was responsible for the Solo UGV which would use SLAM for navigation. This would include identifying a suitable pre-existing simulation environment, required packages, and their containerization for portable execution. He implemented a custom commander node to act as a programmatic controller to move the robot from start to end while measuring useful properties such as time taken, map accuracy and path traversed.

- Rachit Gupta

He was responsible for the UAV with depth camera simulation. This includes importing nav2 artifacts into the PX4 environment and resolving overall dependencies. This was followed by identifying and tuning simulation parameters to meet project requirements, establishing the transform tree for correct data manipulation and developing a perception node that converts input camera data into a navigable occupancy grid.

- Rameez Malik

He was responsible for the UGV that would accept an occupancy grid from the UAV and proceed with navigation without SLAM. This includes bringing up the containerized ROS 2 Jazzy simulation environment on the NCSU VCL VM, adapting the commander node to bootstrap AMCL and log the trajectory for post-run analysis, as well as writing the project-level consolidated README on the main branch that ties the team's workstreams together.

Related Work

The field of search and rescue (SAR) and disaster management has seen a surge in research focusing on the coordination of heterogeneous UAV-UGV frameworks. Current literature surveys emphasize the inherent advantage of these dual-platform systems, with real-time reconnaissance provided by Unmanned Aerial Vehicles (UAVs) complementing the high-utility payload capacities and ground-level endurance of Unmanned Ground Vehicles (UGVs) [1, 2].

Practical research often addresses the challenges of operating in GNSS-denied environments (Global Navigation Satellite Systems) through sophisticated map-merging and cooperative localization techniques. In these scenarios, data from a UAV's monocular SLAM or 3D point cloud is integrated with a UGV's local occupancy grid to enhance pathfinding capabilities [3, 4]. Specific methodologies to achieve this include utilizing attention-based image registration for aerial visual localization to rectify UGV odometry [5], and the development of cost-effective heterogeneous swarms tailored for disaster site assessment [6]. Furthermore, modern exploration techniques have begun to explicitly integrate the unique kinematic constraints of both aerial and ground agents to optimize real-time Next-Best-View planning and multi-target routing [7].

Beyond theoretical advancements, empirical progress has been largely catalyzed by real-world benchmarks and international robotics competitions. The DARPA Subterranean (SubT) Challenge, in particular, served as a vital incubator for robust decentralized SLAM frameworks and unified perception models across diverse hardware [8, 9]. Additionally, initiatives like the EU-based TRIFFID consortium have operationalized these systems by networking multi-robot teams with centralized command stations, leveraging machine learning for real-time environmental analysis to assist first responders [10]. Together, these contributions provide a comprehensive basis for analyzing mapping accuracy, computational requirements, and trajectory efficiency in joint SAR operations.

Technical Approach

4.1 UGV With SLAM

The solo UGV (Unmanned Ground Vehicle) is implemented using the following core packages:

- `nav2` : Navigation and Planning
- `slam_toolbox` : Implement SLAM pipeline (Simultaneous Localization and Mapping)
- `turtlebot3` (Waffle) : Provide simulation environment and robot URDF for manipulation

The execution environment relies on a containerized ROS2 Jazzy environment chosen for its compatibility with the host system specifications (Ubuntu 24.04, 8GB RAM, 8 cores). Docker is utilized to encapsulate the workspace, ensuring strict dependency isolation and reproducible builds without polluting the host OS. Bind mounts are configured to allow persistent storage of generated maps and telemetry data, while custom shell entrypoint scripts automate workspace sourcing and ensure seamless host-to-container code execution. Additionally, graphical passthrough is configured to allow high-fidelity rendering of simulation GUIs directly from the isolated container.

At the simulation layer, the project leverages Gazebo Harmonic via the `nav2_minimal_tb3_sim` package to render the 3D physics environment and the TurtleBot3 hardware model.

To orchestrate the system, a custom ROS2 Python launch file dynamically reads starting and target coordinates from a YAML parameter server, ensuring the robot accurately spawns in the simulated world alongside the visualization tools (RViz) before initializing the autonomous navigation stack.

On the algorithmic front, the UGV employs `slam_toolbox` to process simulated LIDAR and wheel odometry, dynamically generating a live 2D Occupancy Grid of its surroundings in real-time rather than relying on a pre-existing static map.

Programmatic control and telemetry tracking are handled by a custom multi-threaded Python node utilizing the `nav2_simple_commander` API. This node acts as the brain of the operation as it:

- Interfaces directly with Nav2's Action Servers to command the robot to parameterized goal poses
- Calculates the navigation travel time
- Continuously subscribes to the `/odom` topic to log the actual driven trajectory to a CSV file
- Programmatically triggers the `nav2_map_server` utility via subprocesses to capture and serialize the dynamically generated SLAM map (saving both the .pgm and .yaml files) for post-run alignment and quantitative evaluation.

Evaluations were done as follows:

To quantitatively evaluate the system's performance, we focused on two primary domains: navigation efficiency and mapping fidelity with respect to the occupancy grid format. The evaluation scripts automatically parse the generated artifacts such as the trajectory CSV and the serialized SLAM maps to compute the following metrics.

Table 1: Quantitative Performance of Solo UGV with SLAM

Metric	Average Value	Interpretation
Total Traversal Time	30.04 s	The total time taken to explore and reach the goal, averaged over 10 runs. Trajectory is shown in figure 2
Map Accuracy (IoU)	18.9%	The percentage of the generated map that perfectly matches the ground truth simulation map boundaries using Intersection over Union. NOTE: IoU is not a reliable metric since it is image based and thus is biased by the thickness of the walls i.e. pixel count. As seen in the overlay figure 3, the two maps align almost perfectly, and thus the RMSE error value is a much better indicator of SLAM-generated map quality.
Map Error (RMSE)	0.007 m	The average distance (in meters) that the SLAM walls deviated from the true walls using Root Mean Square Error.

Map Registration is needed since the SLAM-generated Occupancy Grid is created relative to the robot’s initial spawn point rather than the absolute Gazebo world origin. We use a robust two-step map registration process before calculating map metrics :

1. Coarse Alignment: The evaluation script parses the robot’s path to extract the true physical starting coordinates. It computes a translation matrix to perfectly overlay the SLAM map’s (0,0) origin onto the corresponding physical coordinate within the Ground Truth (GT) map.
2. Fine Alignment (ICP): To correct for minor rotational drift and sensor noise accumulated during exploration, the system applies the Iterative Closest Point (ICP) algorithm. Both maps are thresholded to extract the coordinates of lethal obstacles (walls) as 2D point clouds.

Once the maps are perfectly aligned via ICP, the system calculates two distinct indicators of mapping accuracy:

- Root Mean Square Error (RMSE): Serving as the primary indicator of geometric fidelity, the RMSE represents the average spatial deviation (converted from pixels back into physical meters) between the aligned SLAM boundaries and the true walls. A low RMSE demonstrates that the slam_toolbox scan-matching algorithm effectively minimized drift and accurately localized the robot. Since RMSE measures point-to-point Euclidean distance, it is highly robust against discrepancies in line-thickness between the two maps.
- Intersection over Union (IoU): To measure strict area overlap, the aligned floating-point SLAM coordinates are discretized back into an integer pixel grid. The IoU is calculated by dividing the intersection of the two maps (True Positives, where SLAM pixels perfectly overlap Ground Truth pixels) by their union. This provides a strict percentage-based score of how much of the true physical footprint was perfectly captured by the mapping algorithm.

```
.
└─ solo_ugv_project
   ├── config
   │   └─ params.yaml
   ├── launch
   │   └─ ugv_tracker.launch.py
   ├── maps
   │   ├── depot.pgm
   │   ├── depot.yaml
   │   ├── my_custom_map.pgm
   │   ├── my_custom_map.yaml
   │   ├── slam_generated_map.pgm
   │   ├── slam_generated_map.yaml
   │   ├── tb3_sandbox.pgm
   │   ├── tb3_sandbox.yaml
   │   ├── warehouse.pgm
   │   └─ warehouse.yaml
   ├── package.xml
   ├── resource
   │   └─ solo_ugv_project
   ├── run_launch.sh
   ├── setup.cfg
   ├── setup.py
   ├── solo_ugv_project
   │   ├── commander_node.py
   │   └─ __init__.py
   ├── test
   │   ├── test_copyright.py
   │   ├── test_flake8.py
   │   └─ test_pep257.py
   └─ worlds
       └─ tb3_sandbox.sdf.xacro

9 directories, 23 files
```

Figure 1: solo_ugv_project Package Directory Structure

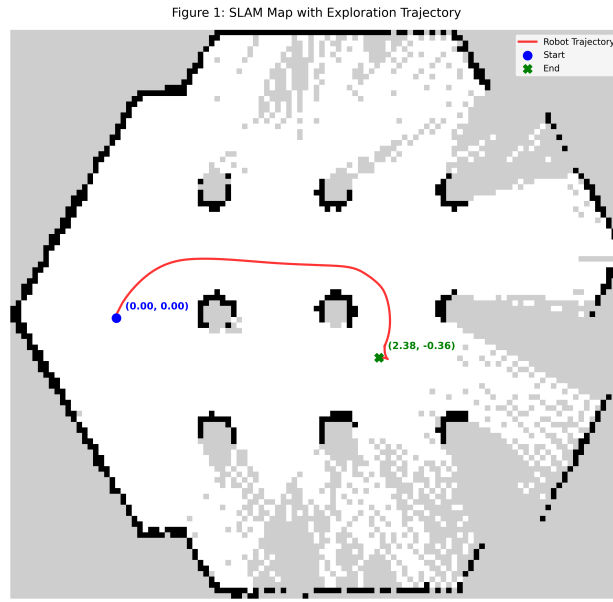


Figure 2: Trajectory Plot on SLAM-generated Map

Figure 2: Ground Truth vs SLAM Map Registration

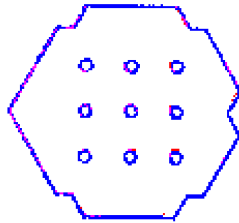
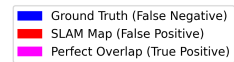


Figure 3: Overlay of SLAM-generated Map on Ground Truth Map

4.2 UGV Without SLAM

The UGV without SLAM is built on the same containerized ROS 2 Jazzy infrastructure as the SLAM-based system, using Docker with bind mounts and entrypoint scripts to enable seamless host-container development. Simulation is handled through modern Gazebo (Harmonic) via the `nav2_minimal_tb3_sim` package, which provides the TurtleBot3 Waffle model along with a physics-based environment. The simulator runs in headless mode, exposing sensor data such as LIDAR and odometry to the ROS 2 graph through `ros_gz_bridge`, while visualization is performed separately using RViz2.

System orchestration is managed by a custom ROS 2 launch file (`ugv_tracker.launch.py`), which initializes the robot using configurable start and goal poses loaded from a YAML parameter file. This launch file integrates the upstream `tb3_simulation_launch.py` to bring up the Nav2 stack and supports a flexible map-loading mechanism: a user-defined occupancy grid (`my_map.yaml`) is used when available; otherwise, the system defaults to a standard sandbox map. Additional environment configurations are required for reliable execution in containerized settings, including dynamically setting `GZ_SIM_RESOURCE_PATH` to resolve missing simulation assets and constraining `gz-transport` communication to loopback via `GZ_IP=127.0.0.1`.

Unlike the SLAM-based approach, this system operates on a pre-built 2D occupancy grid, representing a scenario where environmental mapping is performed externally (e.g., by a UAV) and provided to the UGV. This design simulates a perception-to-action handoff, allowing the UGV to focus purely on localization and navigation. The map is published by the `nav2_map_server`, while `nav2_amcl` performs localization by fusing LIDAR data (`/scan`) and wheel odometry (`/odom`) to estimate the robot’s pose within the fixed map frame.

Navigation is executed through the standard Nav2 pipeline. Global path planning is handled by `nav2_planner` using the NavFn algorithm over layered costmaps, while trajectory execution is performed by `nav2_controller` using a Model Predictive Path Integral (MPPI) controller. Recovery behaviors such as spinning, backing up, and waiting are triggered by `nav2_behaviors` when failures occur, all coordinated through a behavior tree managed by `nav2_bt_navigator`.

At the application level, a custom multi-threaded Python node built on the `nav2_simple_commander` API serves as the central control interface. This node initializes localization with a predefined pose, sends navigation goals via the `/navigate_to_pose` action server, and records performance metrics including travel time and trajectory data. The robot’s path is continuously logged from `/odom` into a CSV file for post-run analysis, and the occupancy grid is captured to support evaluation against ground truth.

The average traversal time value is computed over 10 runs.

Table 2: Quantitative Performance of UGV paired with UAV

Metric	Average Value	Interpretation
Total Traversal Time	14.0 s	The total time taken to explore and reach the goal. Trajectory is shown in figure 4. As compared to the performance in table 1, the UAV assistance provides nearly a 2x speedup in UGV navigation.

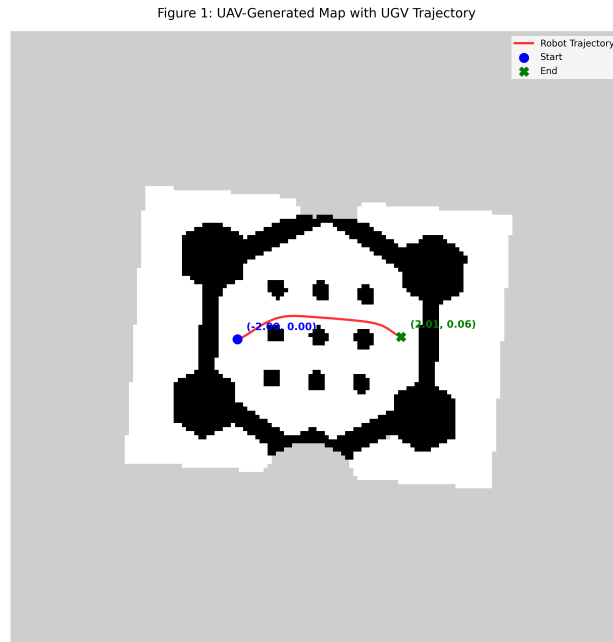


Figure 4: Trajectory Plot on UAV-Generated Map

Figure 2: Ground Truth vs UAV Map Registration

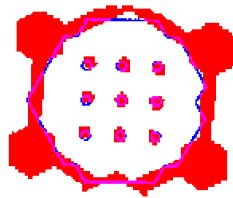
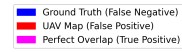


Figure 5: Overlay of UAV-Generated Map on Ground Truth Map

4.3 UAV

The Unmanned Aerial Vehicle (UAV) is simulated using the **PX4 Autopilot** package compatible with the ROS2 Humble installation on Ubuntu 22.04 LTS [11]. PX4 is a modular micro-service microcontroller architecture that runs on the NuttX RTOS. It uses a publish-subscribe middleware called uORB (Mirco Object Request Broker) to communicate between modules. It comes with a **Software-in-the-Loop (SITL)** environment that provides multiple models for advanced drone simulation. We port the uORB messages using the **Micro-XRCE-DDS agent**, a middleware to get the uORB messages into the ROS2 environment and communicate with the microcontroller. The PX4 SITL runs with the QGroundControl, Ground Control Station software that provides critical failsafe and data visualization.

To standardize the environment, turtlebot3_world is ported under world models in PX4 as 'nav' and set using existing parameterization. PX4 ships with a drone model, 'x500_depth' with a forward facing depth camera, Oak-D Lite, with a range of 40cm to 8m. This model is edited to face the camera downwards for aerial mapping in the project.

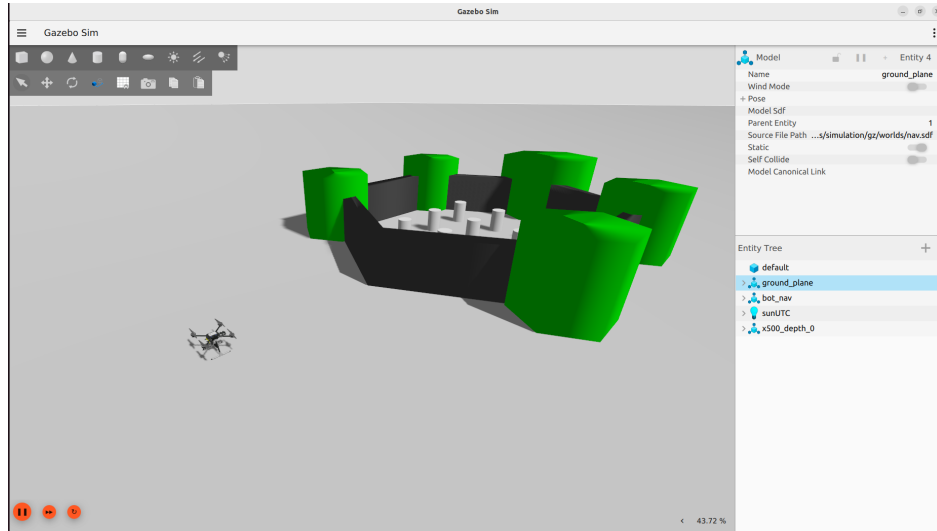


Figure 6: modified x500_depth model with Turtlebot3_world in Gazebo

The autonomous control mode in PX4 is called the offboard mode. A computer can control the drone and provide waypoints to the drone on existing ROS2 topics created by the PX4 environment by issuing a heartbeat signal. Building on top of the **px4_ros_com** package [11], an offboard_control node is created to subscribe to the vehicle local position and issue waypoints to the drone. It uses an FSM to first achieve a target altitude, then moves over the expected UGV environment and holds its position. For our run, the drone took 18.98 seconds to reach the target. In the meantime the camera issues point cloud data (msg_type: PointCloud2) in the Gazebo environment.

For manipulating the Point Cloud data in ROS2, it is ported using a **ROS-GZ bridge** along with the simulation clock, as all the data messages are time-stamped. Furthermore, the point cloud is published in the frame of the camera and the tranformation tree of the simulation is not exposed in ROS2. Hence, nodes are published using static transformer publisher and the **drone_pose** package that create the TF tree in ROS2 based on drone odometry and initial starting position [12] (currently hard-coded, to avoid physically overlapping with the environment model in the physics

simulator).

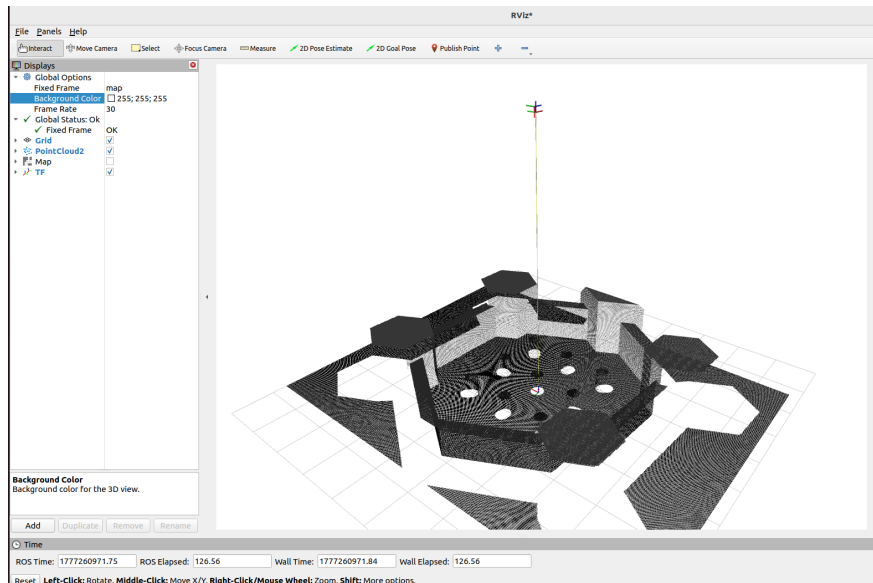


Figure 7: Pointcloud from Gazebo transformed in map frame

Using the TF tree and point cloud data, an Occupancy Grid is published using a modified node of the **pointcloud_to_grid** package [13]. The node is modified to first transform the point cloud into the world frame and then using the heights of the points in the point cloud to determine an obstacle using an obstacle threshold. The node is further modified to center the grid on the location of the drone for accurate grid limits.

A Snapshot of the Occupancy grid is shared with the UGV using a `map_saver_cli` that creates a Portable Gray Map (PGM) for 2D navigation.

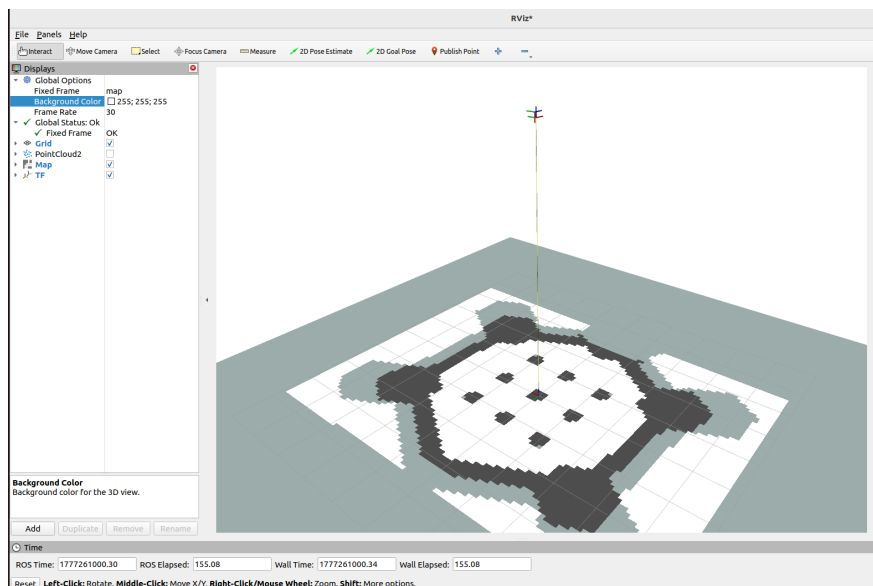


Figure 8: Derived Occupancy Grid for navigation

The average mapping time value is computed over 10 runs. For

Table 3: Quantitative Performance of UAV

Metric	Average Value	Interpretation
Total Traversal Time	16.1 s	The total time taken to arm, takeoff and reach the set waypoint for mapping the target area.

Lessons Learned

The process of working on this project was illuminating in many ways, foremost in terms of scope creep. The initial project proposal was idealistic despite our attempts to make it as grounded and achievable as possible, and we faced multiple challenges in getting the implementation to work. We learned the following lessons:

1. Compute is a Bottleneck

Multiple experiments could not be performed simply due to compute constraints such as insufficient RAM, GPU compatibility and OS compatibility.

We were forced to come up with workarounds and alternative formulations to implement experiments, such as increasing timeouts for action servers and buffer periods for relaying sensor data to ensure that the SLAM to Navigation pipeline does not crash due to throttled compute resources.

2. Robot Initialization

We discovered that `navigator.setInitialPose()` initializes Adaptive Monte Carlo Localization (AMCL) but does not physically teleport the robot in Gazebo, necessitating custom launch files with specific spawn coordinates.

3. Map Alignment

Any SLAM generated map would be relative to the robot's spawn point as origin and thus would require multi-step realignment to compare it with the ground truth map.

4. Time Alignment

Simulation time needs to be aligned across multiple nodes to prevent missed messages and runtime errors which are harder to trace.

5. Coordinates Validation

Any coordinates passed to the robot need to be validated such that they exist in the free space of the occupancy grid to prevent collisions and unreachable targets.

6. Valid Transform Trees

When consuming data from sensors and before performing any manipulation on them, valid transformations need to be registered in the transform tree for valid visualization and navigation

Conclusion

In conclusion, this project successfully developed and evaluated a collaborative UAV-UGV system designed to accelerate emergency response times in dynamic, post-disaster environments. By integrating PX4-Autopilot for aerial reconnaissance and Nav2 for autonomous ground navigation, we established a robust pipeline where overhead mapping directly informs ground-level path planning. Our comparative analysis against a standalone, SLAM-dependent UGV demonstrated the critical value of prior environmental knowledge; the UAV-assisted approach significantly reduced the overall time required by the UGV to reach the destination by 50% by eliminating the need for exploratory ground mapping and preventing route dead-ends. Ultimately, this collaborative framework highlights how bridging aerial data with ground execution can mitigate the risks of navigating unpredictable terrain, ensuring that life-saving relief is deployed more efficiently and reliably when time is of the essence.

References

- [1] I. Munasinghe *et al.*, “A comprehensive review of UAV-UGV collaboration: Advancements and challenges,” *Journal of Sensor and Actuator Networks*, vol. 13, no. 6, 2024.
- [2] S. A. Ghauri *et al.*, “A review of multi-UAV task allocation algorithms for a search and rescue scenario,” *Journal of Sensor and Actuator Networks*, vol. 13, no. 2, 2024.
- [3] M. De Petrillo *et al.*, “Cooperative localization for GNSS-denied subterranean navigation: A UAV-UGV team approach,” *Navigation*, vol. 71, no. 4, 2024.
- [4] G. Christie *et al.*, “Radiation search operations using scene understanding with autonomous UAV and UGV,” arXiv preprint, 2017.
- [5] Y. Zheng *et al.*, “UAVs-based visual localization via attention-driven image registration across varying texture levels,” *Drones*, vol. 8, no. 12, 2024.
- [6] MQP Team, “A heterogeneous swarm solution for disaster reconnaissance,” Worcester Polytechnic Institute, Tech. Rep., 2023.
- [7] ONERA, “Exploration strategies for a heterogeneous UAV-UGV tandem,” in *Proceedings of the European Conference on Mobile Robots (ECMR)*, 2025.
- [8] N. Hudson *et al.*, “Heterogeneous ground and air platforms, homogeneous sensing: Team CSIRO Data61’s approach to the DARPA subterranean challenge,” *Field Robotics*, 2022.
- [9] C. Huang *et al.*, “A heterogeneous unmanned ground vehicle and blimp robot team for search and rescue using data-driven autonomy and communication-aware navigation,” *Field Robotics*, 2022.
- [10] J. Cani, P. Koletsis, K. Foteinos, I. Kefaloukos, L. Argyriou, M. Falelakis, I. D. Pino, A. Santamaria-Navarro, M. Čech, O. Severa, A. Umbrico, F. Fracasso, A. Orlandini, D. Drakoulis, E. Markakis, I. Varlamis, and G. T. Papadopoulos, “Triffid: Autonomous robotic aid for increasing first responders efficiency,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.09379>
- [11] L. Meier and The PX4 Contributors, “PX4 Autopilot.” [Online]. Available: <https://px4.io>
- [12] Graziato Davide, “Px4-ros2-drone composable,” <https://github.com/Fixit-Davide/PX4-ROS2-Drone-Composable>, 2024.

- [13] JKK - Vehicle Industry Research Center, “pointcloud_to_grid ros2 package,” https://github.com/jkk-research/pointcloud_to_grid, 2025.
- [14] I. Munasinghe, A. Perera, and R. C. Deo, “A comprehensive review of uav-ugv collaboration: Advancements and challenges,” *Journal of Sensor and Actuator Networks*, vol. 13, no. 6, 2024. [Online]. Available: <https://www.mdpi.com/2224-2708/13/6/81>
- [15] I. A. Şucan and S. Chitta, “MoveIt!” [Online]. Available: <http://moveit.ros.org>.
- [16] FEMA, “Home,” FEMA, Feb. 2026, [Online]. Available: <https://www.fema.gov>.
- [17] Z. T. Ibrahim and J. He, “Slam technology on disaster response,” *World Journal of Engineering and Technology*, vol. 12, no. 3, pp. 695–714, 2024.
- [18] S. Macenski and I. Jambrecic, “Slam toolbox: Slam for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [19] D. L. A. M. M. F. S. Macenski, T. Moore, “From the desks of ros maintainers: A survey of modern capable mobile robotics algorithms in the robot operating system 2,” *Robotics and Autonomous Systems*, 2023.
- [20] A. Merzlyakov and S. Macenski, “A comparison of modern general-purpose visual slam approaches,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.
- [21] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, “The marathon 2: A navigation system,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [22] ROBOTIS Co., Ltd., “Turtlebot3 open source – robotis e-manual,” <https://emanual.robotis.com/docs/en/platform/turtlebot3/opensource/>, accessed: 2026-04-27.